Chapter 3. Predicting the best states and actions: Deep Q-networks

This chapter covers

- Implementing the Q function as a neural network
- Building a deep Q-network using PyTorch to play Gridworld
- Counteracting catastrophic forgetting with experience replay
- Improving learning stability with target networks

In this chapter we'll start off where the deep reinforcement learning revolution began: DeepMind's deep Q-networks, which learned to play Atari games. We won't be using Atari games as our testbed quite yet, but we will be building virtually the same system DeepMind did. We'll use a simple console-based game called Gridworld as our game environment.

Gridworld is actually a family of similar games, but they all generally involve a grid board with a player (or agent), an objective tile (the "goal"), and possibly one or more special tiles that may be barriers or may grant negative or positive rewards. The player can move up, down, left, or right, and the point of the game is to get the player to the goal tile where the player will receive a positive reward. The player must not only reach the goal tile but must do so following the shortest path, and they may need to navigate through various obstacles.

3.1. The Q function

We will use a very simple Gridworld engine that's included in the GitHub repository for this book. You can download it at http://mng.bz/JzKp in the Chapter 3 folder.

The Gridworld game depicted in figure 3.1 shows the simple version of Gridworld we'll start with; we'll progressively tackle more difficult variants of the game. Our initial goal is to train a DRL agent to navigate the